

Lab3：陷阱(Traps)和延迟分配(Lazy Allocation)

实验一：陷阱(Traps)

本实验将探讨如何实现陷阱的系统调用。实验中，我们将进行一个有关堆栈的小练习，然后完成有关用户级陷阱处理的实验。

一、什么是陷阱

计算机运行模式分为用户态 (user) 和内核态 (kernel) 两种，用户空间包含应用程序的代码和数据，内核空间包含操作系统的核心代码和数据。操作系统在内核态模式中运行，可以执行所有指令，并调用所有硬件资源；用户程序在用户态模式中运行，只能使用有限的指令，无法直接访问硬件。有三种事件会导致CPU搁置指令的执行，强制将控制权转移到处理该事件的特殊位置上，执行陷入机制

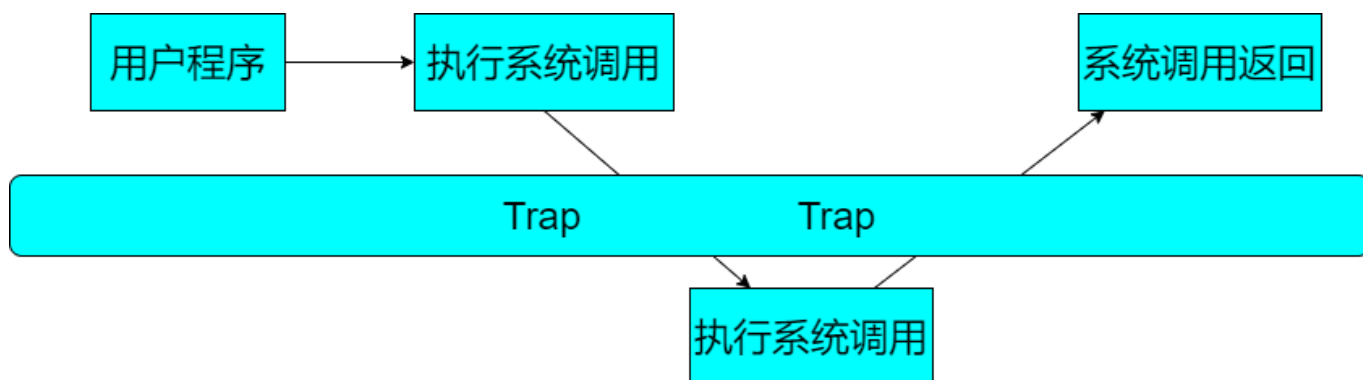
陷入内核态：

- 系统调用 (用户态需要系统调用函数时) 异
- 常 (系统执行异常，如除以0、page fault)
- 中断 (设备中断)

xv6中使用trap作为上述情况的通用术语，traps也称为中断陷阱，是一种特殊的中断，在系统调用和中断处理中具有关键作用，当用户程序执行流到特定指令时主动触发，帮助执行流程从用户态陷入内核态，将控制权转移给操作系统，使得用户可以调用硬件，访问所有指令以获得操作系统提供的服务，完成服务后回到用户态继续执行，可以用于应对诸如系统调用, 虚拟内存页缺失, 硬件信号等等功能。

可能存在：

- 从用户态陷入：可能存在恶意程序，数据需要谨慎对待，需要先保存线程/进程再进行恢复
- 从内核态陷入：比较简单



当发生系统调用时，用户态的程序发起系统调用。因为系统调用中牵扯特权指令，用户态程序权限不足，因此会中断执行，也就是 Trap (Trap 是一种中断)。发生中断后，当前 CPU 执行的程序会中断，跳转到中断处理程序。内核程序开始执行，也就是开始处理系统调用。内核处理完成后，主动触发 Trap，这样会再次发生中断，切换回用户态工作。

二、Risc-v寄存器

寄存器	接口名称	描述	在调用中是否保留?
Register	ABI Name	Description	Preserved across call?
x0	zero	Hard-wired zero 硬编码 0	—
x1	ra	Return address 返回地址	No
x2	sp	Stack pointer 栈指针	Yes
x3	gp	Global pointer 全局指针	—
x4	tp	Thread pointer 线程指针	—
x5	t0	Temporary/alternate link register 临时寄存器	No / 备用链接寄存器
x6-7	t1-2	Temporaries 临时寄存器	No
x8	s0/fp	Saved register/frame pointer 保存寄存器	Yes / 帧指针
x9	s1	Saved register 保存寄存器	Yes
x10-11	a0-1	Function arguments/return values 函数参数	No / 返回值
x12-17	a2-7	Function arguments 函数参数	No
x18-27	s2-11	Saved registers 保存寄存器	Yes
x28-31	t3-6	Temporaries 临时寄存器	No
f0-7	ft0-7	FP temporaries 浮点临时寄存器	No
f8-9	fs0-1	FP saved registers 浮点保存寄存器	Yes
f10-11	fa0-1	FP arguments/return values 浮点参数/返回值	No
f12-17	fa2-7	FP arguments 浮点参数	No
f18-27	fs2-11	FP saved registers 浮点保存寄存器	Yes
f28-31	ft8-11	FP temporaries 浮点临时寄存器	No

第一列：寄存器名字

并不是很重要，它唯一重要的场景是在 RISC-V 的 Compressed Instruction 中。

第二列：ABI 名字

不仅是因为这样描述更清晰和标准，同时也因为在写汇编代码的时候使用的也是ABI名字。

第四列：寄存器在函数调用的时候是否保存

举例：ra 寄存器是 不保留，是因为当函数 a 调用函数 b 的时候，b 会重写 ra 寄存器，存放 a 的地址用作返回。

相关的汇编指令

- csrrw (CSR Read)

csrrw rd, csr, rs

从控制状态寄存器 (csr) 中读取当前值，并将其写入目标寄存器 (rd)，同时将目标寄存器 (rs) 中的值写入控制状态寄存器 (csr')。

- sd (Store Doubleword)

sd rs, offset(rd) 将源寄存器 (rs) 中的64位双字的数据存储到目标地址计算出的内存位置 (rd 寄存器的内容加上偏移量offset) 中。

- csrr (CSR Read/Write)

csrr rd, csr 从控制和状态寄存器 (csr) 中读取当前值，存储到目标寄存器 (rd) 中。

- ld (Load Doubleword)

ld rd, offset(rs) 将从源地址计算出的内存位置 (rs 寄存器的内容加上偏移量offset) 中加载的值存储到目标寄存器 (rd) 中。

- csw (CSR Write) csw csr, rs

将源寄存器 (rs) 中的值写入控制状态寄存器 (csr) 。

- sfence.vma

sfence.vma rs1, rs2

刷新虚拟内存地址， rs1 指定要刷新的虚拟地址， rs2 指定额外的刷新条件，通常是0

- jr (Jump Register)

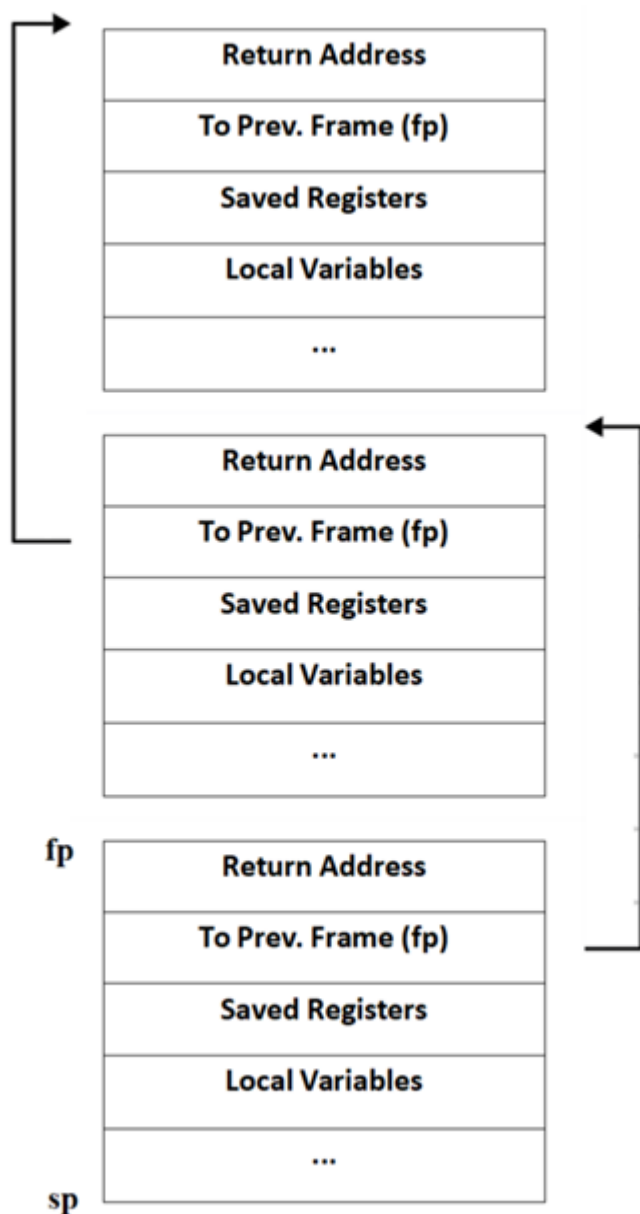
jr rs

跳转到寄存器 (rs) 中存储的地址。

- sret (Supervisor Return)

从异常处理中返回到用户态。

三、栈和栈帧



- 栈 (Stack)

栈是一种先进后出的数据结构，用于存储程序运行时临时创建的数据，包括函数调用过程中的局部变量、参数和返回地址等信息。

- 栈帧 (Stack Frame)

每当一个函数被调用时，都会在栈上分配一个新的区域，这个区域称为栈帧。栈帧包含了该函数的局部变量、传递给函数的参数以及一个返回地址等，这个返回地址指向函数执行完成后应该继续执行的指令位置。

- 函数调用和返回

- 调用：

当函数A调用函数B时，函数A的当前执行位置（即返回地址）和栈指针的状态会被保存起来，然后栈指针移动到新的位置以容纳函数B的栈帧，函数B开始执行。

- 返回：

当函数B执行完毕后，它会从自己的栈帧中读取返回地址，并将栈指针移回函数A的栈帧。然后，

程序通过跳转到这个返回地址继续执行函数A。

- **Return Address**
存储函数调用返回后应该跳转的地址，表明调用函数之后，程序应该回到哪里继续执行。
- **To Prev. Frame (fp)**
指向前一个栈帧的帧指针，用于维护栈的链表结构，函数返回时可以正确地回到调用它的函数。
- **Saved Registers**
保存调用函数时的一些重要寄存器的值，在函数返回时可以恢复这些寄存器的原始值。
- **Local Variables**
存储函数执行过程中所需的局部变量。
- **fp**
帧指针，指向当前栈帧的起始位置。
- **sp**
栈指针，当前栈顶的位置。

栈是从高地址向低地址增长的，当我们新调用一个函数时，会在当前栈帧的更低地址处分配一个新的栈帧。函数执行完毕后，其栈帧将会被销毁，计数器重新跳转到调用该函数的位置的下一条指令继续执行。这个跳转的位置可以通过读取返回地址来获得，该地址保存在栈帧的某个位置。

！！！注意返回地址位于栈帧帧指针的固定偏移(fp-8)位置，并且保存的帧指针位于帧指针的固定偏移(fp-16)位置

XV6在内核中以页面对齐的地址为每个栈分配一个页面。你可以通过PGROUNDDOWN(fp)和PGROUNDUP(fp) (参见kernel/riscv.h) 来计算栈页面的顶部和底部地址。

任务1：Risc-v assembly

实验要求

xv6仓库中有一个文件user/call.c。执行make fs.img编译它，并在user/call.asm中生成可读的汇编版本。阅读call.asm中函数g、f和main的代码。这个实验是一个学习性质的实验，设计这个实验本质上是想让我们回顾和熟悉一下RISC-V汇编语言和Calling Convention。这个实验就是阅读一些汇编语言程序并回答一些问题

call.c函数

```
// g函数作用，给传入的值加3
int g(int x) {
    return x+3;
}

// f函数就是g函数的直接封装
int f(int x) {
    return g(x);
}
```

```
// 调用f函数，打印两个值
void main(void) {
    printf("%d %d\n", f(8)+1, 13);
    exit(0);
}
```

可以看到，main函数调用了f函数，f函数又是g函数的简单包装。按照xv6实验指导书的说明，我们使用make fs.img指令可以编译这个C代码文件，并可以产生一个可读的汇编语言文件。基于此汇编代码，回答下述问题：

```
//编译user/call.c，在控制台输入：
$ ls
$ make fs.img
```

得到user/call.asm，观察g、f和main函数部分对应的代码

```
void main(void) {
1c: 1141          addi    sp,sp,-16
1e: e406          sd     ra,8(sp)
20: e022          sd     s0,0(sp)
22: 0800          addi    s0,sp,16
printf("%d %d\n", f(8)+1, 13);
24: 4635          li     a2,13 //将13放置到a2寄存器
26: 45b1          li     a1,12 //将12放置到a1寄存器
28: 00000517      auipc   a0,0x0 //a0 = pc = 0x28
2c: 7b050513      addi    a0,a0,1968 # 7d8 <malloc+0xea> //a0 + 1952 =
0x28 + 0x7a0 = 0x7c8
30: 00000097      auipc   ra,0x0 //ra = pc = 0x30
34: 600080e7      jalr    1536(ra) # 630 <printf> //pc = ra + 1528 =
0x30 + 0x5f8 = 0x628
                                     //ra = pc + 4 = 0x38

exit(0);
38: 4501          li     a0,0
3a: 00000097      auipc   ra,0x0
3e: 27e080e7      jalr    638(ra) # 2b8 <exit>
```

Q1:哪些寄存器用来存放函数所用的参数？比如在main函数中调用printf时13这个参数是在哪个寄存器中传递进去的？

Q2：main函数中对f函数调用的汇编代码在哪里？对g函数的调用又在哪里？(提示：编译器可以对函数进行内联)

Q3：printf函数的地址是什么？

Q4：在main函数中执行完跳转到printf函数的jalr指令之后，ra寄存器的值是什么？

任务2：Backtrace

实验要求

本实验要实现一个内核函数 `backtrace`，该函数用来回溯调用者的函数调用链，将所有的调用者的 `pc` 打印出来，也就子函数的 `ra`。只需要从当前 `Stack Frame` 开始，通过 `Prev` 找到调用者的 `Stack Frame`，直至调用链结束为止。

```
backtrace:
0x0000000080002cda
0x0000000080002bb6
0x0000000080002898
```

实验须知：

- 每调用一个函数，函数都会为自己创建一个 `Stack Frame`。
- 栈是从高地址开始向低地址使用。所以栈总是向下增长。
- `Return address`总是会出现在 `Stack Frame` 的第一位，指向前一个 `Stack Frame` 的指针也会出现在栈中的固定位置
- 第一个寄存器是 `SP`（`Stack Pointer`），它指向 `Stack` 的底部并代表了当前 `Stack Frame` 的位置。第二个寄存器是 `FP`（`Frame Pointer`），它指向当前 `Stack Frame` 的顶部。`FP-8` 位是返回地址，`FP-16` 位是上一个 `stack frame` 的 `fp` 地址。保存前一个 `Stack Frame` 的指针的原因是为了让我们能跳转回去。
- `XV6` 在内核中以页面对齐的地址为每个栈分配一个页面。你可以通过 `PGROUNDDOWN(fp)` 和 `PGROUNDUP(fp)`（参见 `kernel/riscv.h`）来计算栈页面的顶部和底部地址。

Step1：首先，要获取到当前 `Stack Frame` 的 `fp`，在 `kernel/riscv.h` 中添加函数：

```
// kernel/riscv.h
static inline uint64
r_fp()
```

```

{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x) );
    return x;
}

```

这个函数使用内联汇编来读取s0

Step2:kernel/defs.h添加backtrace的声明

```

// kernel/printf.c
void        printf(char*, ...);
void        panic(char*) __attribute__((noreturn));
void        printfinit(void);
void        backtrace(void);

```

Step3: 这个函数就是实现曾经调用函数地址的回溯，这个功能在日常的编程中也经常见到，编译器报错时就是类似的逻辑，只不过题目的要求较为简单，只用打印程序地址，而实际的报错中往往打印程序文件名，函数名以及行号等信息（最后的可选练习就是实现这样的功能）

```

// kernel/printf.c
void
backtrace(void)
{
    // 读出当前FP指针的值，使用上面添加的嵌入式汇编函数
    // 根据FP指针的值计算出栈底位置，这将作为循环结束的条件
    uint64 fp = r_fp();
    uint64 top = PGROUNDUP(fp);
    printf("backtrace:\n");
    // 如果没有到最后一级，则持续向前一级回溯
    while(fp < top) {
        // 从FP-8这个位置取出上一级函数返回地址，打印出来
        uint64 retaddr = fp-8;
        printf("%p\n", *(uint64*)retaddr);
        // 回溯到上一层函数栈帧
        fp = *((uint64*)(fp-16));
    }
}

```

Setp4:此外，实验要求，在发生 panic 时调用 backtrace，因此在 kernel/printf.c 文件中的 panic 函数里添加 backtrace 的函数调用；在 sys_sleep 代码中也添加同样的函数调用

```

// kernel/printf.c
void
panic(char *s)
{
    pr.locking = 0;

```

```

printf("panic: ");
printf(s);
printf("\n");
backtrace();//添加
panicked = 1; // freeze uart output from other CPUs
for(;;)
    ;
}

```

```

// kernel/sysproc.c
uint64
sys_sleep(void)
{
    int n;
    uint ticks0;

    if(argint(0, &n) < 0)
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        if(myproc()->killed){
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    backtrace();//添加
    return 0;
}

```

Step5:测试 在xv6代码库中执行make qemu, xv6中执行bttest, bttest调用sleep(system call) 。依次调用trap.c、syscall.c、sysproc.c。

```
july@july-virtual-machine: ~/桌面/riscv-gnu-toolchain/xv6-labs-2020
july@july-virtual-machine:~/桌面/riscv-gnu-toolchain/xv6-labs-2020$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/start.o kernel/start.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/console.o kernel/console.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/printk.o kernel/printk.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/uart.o kernel/uart.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/kalloc.o kernel/kalloc.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/spinlock.o kernel/spinlock.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/string.o kernel/string.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/main.o kernel/main.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/vm.o kernel/vm.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/proc.o kernel/proc.c
riscv64-linux-gnu-gcc -c -o kernel/switch.o kernel/switch.S
riscv64-linux-gnu-gcc -c -o kernel/trampoline.o kernel/trampoline.S
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/trap.o kernel/trap.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/syscall.o kernel/syscall.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/sysproc.o kernel/sysproc.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
```

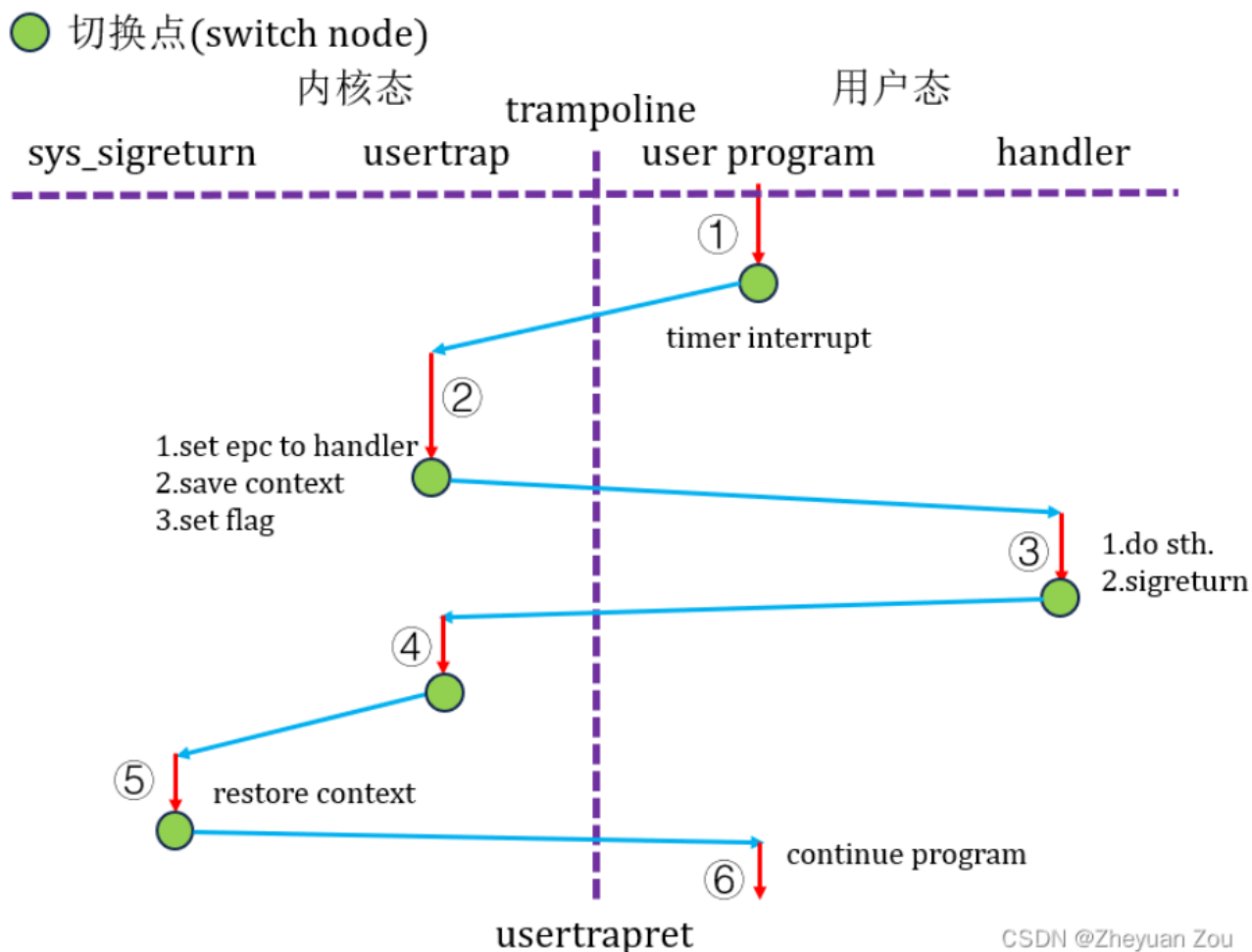
```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ bttest
backtrace:
0x0000000000002dee
0x0000000000002c50
0x00000000000028d0
$
```

任务3：Alarm

实验要求

本实验你将向XV6添加一个特性，在进程使用CPU的时间内，XV6定期向进程发出警报。这对于那些希望限制CPU时间消耗的受计算限制的进程，或者对于那些计算的同时执行某些周期性操作的进程可能很有用。更普遍的说，你将实现用户级中断/故障处理程序的一种初级形式。例如，你可以在应用程序中使用类似的一些东西处理页面故障。

根据xv6实验指导书的介绍，本实验需要实现两个系统调用sigalarm和sigreturn，实现一个进程执行时定时打断转去执行其他函数之后返回的功能。



Step1:实现系统调用sigalarm 这个系统调用可以设置一个进程被时钟打断的间隔(Inteval)和响应函数(handler), 一经设定, 这个进程就会以Inteval的时间间隔调用处理函数handler, 从而进入上图的完整调用流程。因为时间间隔和调用函数handler都是与进程状态强相关的, 所以我们将其保存在struct proc中, 经过修改之后的proc结构体如下(新添加的域都以尖括号的形式注释)

```
//kernel/proc.h
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    struct proc *parent;           // Parent process
    void *chan;                     // If non-zero, sleeping on chan
    int killed;                     // If non-zero, have been killed
    int xstate;                     // Exit status to be returned to parent's wait
    int pid;                        // Process ID

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;                  // Virtual address of kernel stack
    uint64 sz;                      // Size of process memory (bytes)
    pagetable_t pagetable;          // User page table
    struct trapframe *trapframe;    // data page for trampoline.S
    struct context context;         // swch() here to run process
    struct file *ofile[NOFILE];    // Open files
    struct inode *cwd;              // Current directory
```

```

char name[16];                // Process name (debugging)

int interval;                 // <触发alarm handler的时间间隔>
uint64 handler;               //<handler在用户态的虚拟地址>
int ticks;                    //<计时器，记录从上次触发handler到现在经过的ticks数量>
struct trapframe* trapframe2; // <在发生sigalarm调用时，用于保存之前的trapframe>
};

```

实现sys_sigalarm非常简单，只是将用户传进来的参数记录到进程结构体中即可

```

//kernel/sysproc.c
uint64
sys_sigalarm(void)
{
    int interval;
    uint64 handler;
    struct proc* p;

    if(argint(0, &interval)<0 || argaddr(1, &handler)<0)
        return -1;

    p = myproc();
    p->interval = interval;
    p->handler = handler;

    return 0;
}

```

在内核中实现了sigalarm系统调用之后，用户可能会在它的某个程序执行期间调用sigalarm系统调用，从而完成中断间隔时间和中断处理函数在内核的注册。一经注册，内核就开始对时间间隔进行统计，并开始周期性地调用这个中断处理函数了。

Step2:修改usertrap()函数 如同图中的①所示，一个进程在执行时，定时器可能会在其中某个不确定的时刻触发一个时钟中断。这时候将会进入系统调用的相应流程，首先经由trampoline进入usertrap函数，也就是上图中的②，在这里我们将对这个时钟中断进行处理，并适时地进行alarm响应过程，

```

// kernel/trap.c
if(p->killed)
    exit(-1);

// give up the CPU if this is a timer interrupt.
if(which_dev == 2) {
    if(p->interval <= 0)
        goto yield;

    p->ticks++;
    if(p->ticks == p->interval) {
        *p->trapframe2 = *p->trapframe;
        p->trapframe->epc = p->handler;
    }
}

```

```

    }

yield:
    yield();
}

```

usertrap的修改逻辑，这里做的事情就是记录经过的时间，如果时间达到了我们设定的间隔，则触发alarm响应过程。首先将trapframe完全保存，然后设置trapframe中的epc寄存器的值设置为handler的虚拟地址，使其可以经由usertrapret返回到用户态下的handler函数中去。别忘了还要设置标志位InHandler，这可以避免重入(re-enter)handler而造成的错误。

经过我们的设置，现在回到了上图中的③，在这里会执行一些用户定义的动作，并在最后执行sigreturn系统调用

Step3：sys_sigreturn系统调用 经过usertrap的转发(上图中的④)之后进入内核的sys_sigreturn系统调用，sys_sigreturn的实现非常简单，只是简单的恢复现场并重置计数器，如上图中的⑤所示

```

//kernel/sysproc.c
uint64
sys_sigreturn(void)
{
    struct proc* p;

    p = myproc();
    *p->trapframe = *p->trapframe2;

    /** 为下一次回调handler函数做准备 */
    p->ticks = 0;

    return 0;
}

```

恢复了最早程序在执行用户态程序时的现场，并将计时器和标志全部初始化，这样进程就可以成果响应下一次alarm了:)，接下来经过usertrapret的恢复，现在进程又回到了最早的程序中，且现场没有受到任何影响，如图中的⑥所示。

Step4:实验

alarmtest测试

```
july@july-virtual-machine: ~/桌面/riscv-gnu-toolchain/xv6-labs-2020
july@july-virtual-machine:~/桌面/riscv-gnu-toolchain/xv6-labs-2020$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/console.o kernel/console.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/printf.o kernel/printf.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/uart.o kernel/uart.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/spinlock.o kernel/spinlock.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/proc.o kernel/proc.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/trap.o kernel/trap.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/syscall.o kernel/syscall.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/sysproc.o kernel/sysproc.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/fs.o kernel/fs.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/sleeplock.o kernel/sleeplock.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/file.o kernel/file.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/pipe.o kernel/pipe.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/exec.o kernel/exec.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-
-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/sysfile.o kernel/sysfile.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/start.o kernel/console.o kernel/
```

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ alarmtest
test0 start
.....alarm!
test0 passed
test1 start
...alarm!
...alarm!
...alarm!
...alarm!
...alarm!
...alarm!
...alarm!
...alarm!
...alarm!
test1 passed
test2 start
.....alarm!
test2 passed
```

usertest测试

关闭终端，重新打开，输入make qemu

输入 usertests

```
exec usertest failed
$ usertests
usertests starting
test execout: OK
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
test reparent2: OK
test pgbug: OK
test sbrkbugs: usertrap(): unexpected scause 0x000000000000000c pid=3237
sepc=0x0000000000005406 stval=0x0000000000005406
usertrap(): unexpected scause 0x000000000000000c pid=3238
sepc=0x0000000000005406 stval=0x0000000000005406
OK
test badarg: OK
test reparent: OK
test twochildren: OK
test forkfork: OK
test forkforkfork: backtrace:
0x0000000000002dd0
0x0000000000002c32
0x00000000000028cc
backtrace:
0x0000000000002dd0
0x0000000000002c32
0x00000000000028cc
```

成绩评分

关闭终端，重新打开，输入./grade-lab-traps alarmtest

```
\n\n== Test    alarmtest: test0 ==  
    alarmtest: test0: OK  
== Test    alarmtest: test1 ==  
    alarmtest: test1: OK  
== Test    alarmtest: test2 ==  
    alarmtest: test2: OK
```

实验二：Lazy Allocation

这个实验将使你熟悉内存分配机制。你将实现页表分配方式从即时分配到懒分配的切换。

那么，内存分配机制为什么需要设置成 Lazy allocation 呢？首先，内核为一个大请求分配和映射内存可能需要很长时间。例如，考虑一个千兆字节由 262144 个 4096 字节的页面组成，即使每个页面都很便宜，分配的次数也非常多。其次，有些程序分配的内存比实际使用的还要多（例如，为了实现稀疏数组），或者在使用前就提前分配内存，这样会导致有些进程申请的内存可能一直不会使用。以上情况都会导致 CPU 资源不必要占用或内存资源浪费。懒分配模式就是为解决上述问题而产生的。

实验原理

懒分配的定义

计算机中运行着很多进程，进程会提出内存分配申请。如果进程一提出内存申请，计算机就满足这一申请为它分配物理内存页，则称为 Eager Allocation 即即时分配。然而，如果计算机为进程分配内存时，只增大进程的内存空间字段值，但并不立即进行实际的物理内存页分配；之后当该内存段确实需要使用时，会因找不到内存页，产生缺页中断（page fault interrupt），这时再进行物理内存的分配，这一分配方式便是 Lazy Allocation 即懒分配。

进程空间

每个进程有独立的虚拟地址空间，进程访问的虚拟地址并不是真正的物理地址，但进程可通过自身页表中虚拟地址与物理地址建立的映射获得真正的物理地址。xv6 使用页表分隔不同进程的地址空间，并把它们多路复用到单一的物理内存。

xv6 中的虚拟内存管理

在 xv6 中，用户程序运行和内核运行的时候，使用的是不同的页表。

xv6 的两个页表（全局内核页表 and 用户页表）都是完成虚拟地址到物理地址的映射的记录。但是为了安全，操作系统在内核态当中不能直接访问用户态的地址空间，所以内核态不能直接翻译用户态的虚拟地址，需要遍历用户态的用户页表（在 PCB 当中），完成虚拟地址（va）到物理地址（pa）的转换。

用户程序页表仅包含自身的代码和数据的虚实地址映射，内核代码和数据不包含在内。当进程申请更多内存的时候，xv6 首先用 kalloc 分配一个物理页。然后调用 mappages 函数把这个物理页的 PTE 加到进程的页表里。xv6 会设置该 PTE 对应的标志位(W,X,R,U,V)。

实验目的

通过将 xv6 的页表分配方式从即时分配修改为懒分配，熟悉系统调用、内存管理的相关知识及页表分配的具体实现过程。

实验要求

任务一：取消系统调用中原有的页面分配功能

在本任务中，你将取消 sbrk 系统调用中原有的页面分配功能。

你需要完成的有：

修改 kernel/sysproc.c 中的 sys_sbrk()，取消 Eager Allocation，只增大进程的内存空间字段值，但并不立即进行实际的物理内存页分配。

一些提示：

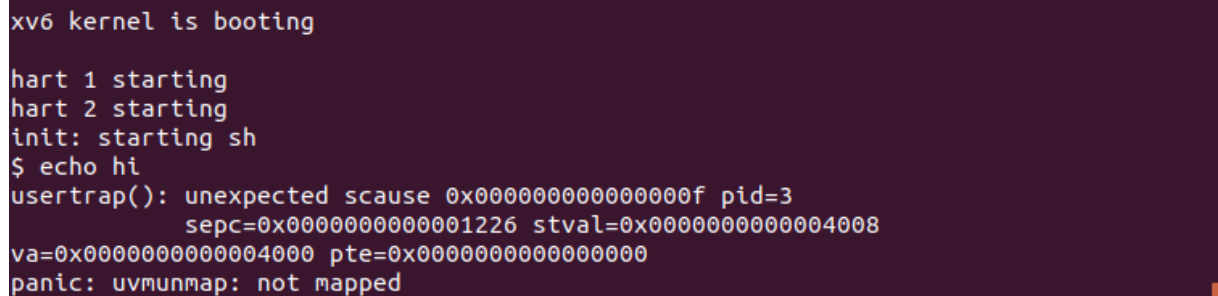
在 xv6 操作系统中，sys_sbrk 函数实现了 sbrk 系统调用。sbrk 系统调用于改变程序间断点的位置和数据段长度，实现虚拟内存到物理内存的映射。具体实现中，sys_sbrk 函数会将进程的内存大小增加 n 个字节，并分配内存，然后返回新分配区域的起始地址值。

```
# kernel/sysproc.c
uint64
sys_sbrk(void) //修改前的 sys_sbrk 函数
{
    uint64 addr; // 定义一个整数变量 addr，用于存储当前堆的末尾地址
    int n; // 定义一个整数变量 n，用于存储要增加或减少的字节数

    if(argint(0, &n) < 0) // 从系统调用的参数中获取第一个整数参数 n
        return -1; //如果获取参数失败（返回值小于 0），则函数返回-1，表示错误
    addr = myproc()->sz; // 获取当前进程数据段的末尾地址，存储在 addr 中
    if(growproc(n) < 0) // 调用 growproc(n) 来实际调整进程的数据段大小
        return -1;
    return addr; // 返回数据段堆末尾地址
}
```

最后进行测试：

在 xv6 环境下执行 echo hi 命令并检查输出。



```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ echo hi
usertrap(): unexpected scause 0x000000000000000f pid=3
          sepc=0x0000000000001226 stval=0x0000000000004008
va=0x0000000000004000 pte=0x0000000000000000
panic: uvmunmap: not mapped
```

执行 echo hi 会触发缺页中断。取消立即分配后，因为没有从虚拟地址映射到物理地址（not mapped），使得 xv6 内核 panic 了。

"usertrap()：....."信息来自 trap.c 中的用户陷阱处理程序，它捕捉到了缺页中断。

任务二：实现懒分配功能（以 echo hi 命令测试）

在本任务中，你将修改代码以响应来自用户空间的页面错误。方法是新分配一个物理页面并映射到发生错误的地址，然后返回到用户空间，让进程继续执行，以使 echo hi 正常工作。

你需要完成的有：

1. 处理中断，分配内存。
2. 处理步骤 1 的相关修改后引发的当前 uvmunmap()导致系统 panic 崩溃的问题。

一些提示：

usertrap 函数是 xv6 中处理从用户态陷入内核态的中断和异常的核心部分，确保系统能够正确响应和处理各种异常和中断情况，维持系统的稳定和正常运行。

从任务一的测试结果可知，目前的修改触发了代码号为 15 的 Store page fault 缺页中断。而原本的 usertrap 函数中不存在页面故障的中断处理逻辑。因此需要在 usertrap 中添加额外的中断处理逻辑，通过在故障地址映射新分配的物理内存页来响应来自用户空间的页面错误，然后返回用户空间，让进程继续执行。

```

# kernel/trap.c

void usertrap(void) { //修改前的 usertrap 函数
    int which_dev = 0;

    if ((r_sstatus() & SSTATUS_SPP) != 0) // 检查 sstatus 寄存器中的
SPP 位，以确保陷入是从用户模式发生的。如果不是，则会触发 panic
        panic("usertrap: not from user mode");

    w_stvec((uint64)kernelvec); // 此时已处于内核态。将中断向量设置为
kernelvec，接下来的中断和异常将发送到 kernelvec 进行处理

    struct proc *p = myproc(); // 获取当前进程

    p->trapframe->epc = r_sepc(); // 将当前的用户程序计数器 (r_sepc())
保存到进程的 trapframe 中，以便在处理完中断或异常后能够正确返回用户态继续执行


    if (r_scause() == 8) { // 检查是否为系统调用
        // 系统调用处理
        if (p->killed)
            exit(-1);

        p->trapframe->epc += 4; // sepc 指向 ecall 指令，但我们需要返回到下
一条指令

        intr_on(); // 中断会改变 sstatus 等寄存器，因此在处理完这些寄存器之前
不要启用中断

        syscall(); // 调用系统调用处理函数
    } else if ((which_dev = devintr()) != 0) { // 处理设备中断
        // ok
    } else { // 处理未预期的 scause
        printf("usertrap(): unexpected scause %p pid=%d\n",
r_scause(), p->pid); //打印错误信息
        printf("                sepc=%p stval=%p\n", r_sepc(),
r_stval());
        p->killed = 1; // 标记进程为已杀死
    }

    if (p->killed) // 处理被杀死的进程
        exit(-1); // 如果进程被杀死，则退出

    if (which_dev == 2) //处理定时器中断
        yield(); // 如果这是一个定时器中断，则调用 yield 函数放弃 CPU，使其他进
程有机会运行

    usertrapret(); // 返回用户态
}

```

uvmunmap()是在释放内存时调用的函数。经过上述修改，由于懒分配并不立即分配进程的物理内存（但会有虚拟地址），对于用户进程的虚拟空间，会出现有的虚拟内存未映射到实际的物理内存的情况，即页表中的 PTE 是无效的。而原本的分配模式下，uvmunmap()函数对于虚拟地址对应的 PTE 无效的情况，会引发 panic。我们需要对它进行修改，使得此时不引发 panic。

最后进行测试：

重新编译，启动 xv6，此时应能够成功运行 echo hi 命令。

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ echo hi
hi
$
```

任务三：通过所有懒惰测试和用户测试

内存分配机制修改后，各种命令的执行过程中，可能引起旧的分配机制下不会出现的各种报错。为了通过全部测试，你需要进行相应处理，使得懒分配模式下这些报错同样不出现。

你需要完成的有：

1. 处理 sys_sbrk()函数的参数为负数的情况。
2. 处理虚拟地址对应的 PTE 未分配导致的 panic。
3. 实现 fork()中的父子内存拷贝的懒分配。
4. 处理 usertrap 函数的页面故障处理中，虚拟地址超过 p->sz 或低于用户栈，以及物理页分配失败的情况。
5. 处理进程向系统调用（如读取或写入等）传递有效地址，但该地址的内存尚未分配的情况。

一些提示：

uvmunmap()调用 walk()来获取 PTE。walk()可以根据虚拟地址 va 得到其相应页表中对应的 PTE。xv6 采用三级页表结构，由于我们实现了懒分配，即未分配 va 对应的物理内存，因此 L2 或 L1 或 L0 的 PTE 都可能不存在，这会导致 walk()返回 0，即(pte = walk(pagetable, a, 0)) == 0 成立而引发 panic。但是未分配 PTE 是由于本实验实现了懒分配，不应触发错误。

fork()是通过 uvmcopy() 函数来进行父进程页表（即用户空间）向子进程的拷贝的。

read 或 write 最终会调用 copyin() 和 copyout() 进行用户空间到内核空间的读写。而这两个函数由虚拟地址 va 到物理地址 pa 的转换是通过 walkaddr()函数完成的。

walkaddr()函数通过调用 walk() 根据虚拟地址 va 和页表 pagetable 得到页表项 PTE。获得 PTE 后，walkaddr()会对其进行错误检测和相应处理，当 PTE 不存在、PTE 无效（PTE_V 标志位为 0）、PTE_U 标志位为 0 时，都会返回 0 表示物理地址地址获取失败。

```

# kernel/vm.c
uint64
walkaddr(pagetable_t pagetable, uint64 va) 、
{
    pte_t *pte;
    uint64 pa;

    if(va >= MAXVA) // 检查虚拟地址是否超过最大值，超过则失败
        return 0;

    // 获取虚拟地址 va 对应的页表项
    pte = walk(pagetable, va, 0);

    if(pte == 0) //PTE 不存在
        return 0;
    if((*pte & PTE_V) == 0) //PTE 无效 (PTE_V 标志位为 0)
        return 0;
    if((*pte & PTE_U) == 0) // PTE_U 标志位为 0 即页表项不允许用户态访问
        return 0;

    // 获取页表项对应的物理地址并成功返回
    pa = PTE2PA(*pte);
    return pa;
}

```

最后进行测试：

在你的主要 xv6 目录中，输入 make grade 进行测试。

```

== Test running lazytests ==
$ make qemu-gdb
(4.3s)
== Test    lazy: map ==
    lazy: map: OK
== Test    lazy: unmap ==
    lazy: unmap: OK
== Test usertests ==
$ make qemu-gdb
(107.6s)
== Test    usertests: pgbug ==
    usertests: pgbug: OK
== Test    usertests: sbrkbugs ==
    usertests: sbrkbugs: OK
== Test    usertests: argptest ==
    usertests: argptest: OK
== Test    usertests: sbrkmuch ==
    usertests: sbrkmuch: OK
== Test    usertests: sbrkfail ==
    usertests: sbrkfail: OK
== Test    usertests: sbrkarg ==
    usertests: sbrkarg: OK
== Test    usertests: stacktest ==
    usertests: stacktest: OK
== Test    usertests: execout ==
    usertests: execout: OK
== Test    usertests: copyin ==
    usertests: copyin: OK
== Test    usertests: copyout ==
    usertests: copyout: OK
== Test    usertests: bigfile ==
    usertests: bigfile: OK
== Test    usertests: dirfile ==
    usertests: dirfile: OK
== Test    usertests: iref ==
    usertests: iref: OK
== Test    usertests: forktest ==
    usertests: forktest: OK
== Test time ==
time: OK
Score: 119/119

```

可供参考的实验代码

实验一：取消系统调用中原有的页面分配功能

更新后的 `sys_sbrk` 只是将进程的内存大小 (`myproc()->sz`) 增加 `n` 个字节，然后返回新分配区域的起始地址值，而不应该分配内存——因此应该删除对 `growproc()` 的调用，但仍需增加进程的大小。

原本调用的 `growproc()` 函数用于实际调整进程的数据段大小，并检查是否成功。注释掉 `growproc()` 后，直接修改 `myproc()->sz` 的值，执行 `myproc()->sz += n` 直接增加或减少当前进程的堆大小。

```

# kernel/sysproc.c
uint64
sys_sbrk(void) //修改后的 sys_sbrk 函数
{
    uint64 addr;
    int n;
    if(argint(0, &n) < 0)
        return -1;
    addr = myproc()->sz;
    myproc()->sz += n; // 直接增加当前进程的数据段大小，并停止调用
growproc
    //if(growproc(n) < 0) // 原本应该调用 growproc(n) 来实际调整进程的数
据段大小
    // return -1;
    return addr; // 返回调整前的数据段末尾地址
}

```

实验二：实现懒分配功能（以 echo hi 命令测试）

1. 在 usertrap 函数中，参数 r_scause()保存中断原因。根据 RISC-V scause 代码号，13 号（Load page fault）和 15 号（Store page fault）都属于页面故障，需要添加 r_scause=13 或 15 的分支判断。参数 r_stval()保存导致页面错误的虚拟地址，分配时需要取出该地址。

```

# kernel/trap.c

void usertrap(void) { //修改后的 usertrap 函数

    .....

    // 添加 r_scause = 13 或 15 的情况
    else if (r_scause() == 13 || r_scause() == 15) {
        char *pa;
        if((pa = kalloc()) != 0) { // 分配物理页
            uint64 va = PGROUNDDOWN(r_stval()); // 引发 page fault 的虚拟
            地址向下取整
            memset(pa, 0, PGSIZE);
            // 进行页表映射
            if(mappages(p->pagetable, va, PGSIZE, (uint64)pa,
            PTE_W|PTE_R|PTE_U) != 0) {
                // 页表映射失败
                kfree(pa);
                printf("usertrap(): mappages() failed\n");
                p->killed = 1;
            }
        } else { // 分配物理页失败
            printf("usertrap(): kalloc() failed\n");
            p->killed = 1;
        }
    } else if ((which_dev = devintr()) != 0) { // 处理设备中断
        // ok
    } else { // 处理未预期的 scause
        printf("usertrap(): unexpected scause %p pid=%d\n",
        r_scause(), p->pid);
        printf("          sepc=%p stval=%p\n", r_sepc(),
        r_stval());
        p->killed = 1; // 标记进程为已杀死
    }

    .....
}

```

2. 对于 `uvmunmap()` 的修改，用 `continue` 语句代替 `panic` 语句即可。

```

# kernel/vm.c

void
uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int
do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            panic("uvmunmap: walk");
        if((*pte & PTE_V) == 0) //V 是 valid 的缩写，值为 0 表示该页表项无效
            continue; //PTE 无效时，不产生 panic，而是跳过
        //panic("uvmunmap: not mapped");
        if(PTE_FLAGS(*pte) == PTE_V)
            panic("uvmunmap: not a leaf");
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}

```

实验三：通过所有懒惰测试和用户测试

1. 参数为负表示内存减少。参考原本的 sys_sbrk()函数中调用的 growproc()函数，与其相似，在懒分配的情况下减少内存同样是将多余的内存进行释放。由于任务一中已停用 growproc()，此处调用 uvmdealloc()函数释放内存。同时添加边界条件判断：由于 addr 和 n 是 int 类型，因此进行防溢出判断 (addr+n>=addr)，避免 addr+n 由正变负造成 p->sz 变小的情况；进一步，要求 addr+n 大于等于 PGROUNDUP(p->trapframe->sp)，防止分配后的地址缩减到用户栈及其以下结构。

```

# kernel/sysproc.c
uint64
sys_sbrk(void) { //第二次修改后的 sys_sbrk 函数
    int addr;
    int n;
    if(argint(0, &n) < 0)
        return -1;
    struct proc* p = myproc();
    addr = p->sz; // 保存原本旧的 sz
    int sz = p->sz;
    if(n >= 0 && addr + n >= addr){
        p->sz += n; // 对于 n 为正数即内存增加的情况，直接增大 p->sz 但不实际分
        配内存
    }else if(n < 0 && addr + n >= PGROUNDUP(p->trapframe->sp)){
        // 对于 n 为负数即内存减少的情况，需要解映射，同时修改解映射后的 p->sz
        p->sz = uvmdealloc(p->pagetable, sz, sz + n);
    }else{
        return -1;
    }
    return addr; // 返回旧地址
}

```

2. 上文说到，未分配 PTE 是由于本实验实现了懒分配，不应触发错误。所以应该跳过此类情况造成的 panic。进一步修改 `uvmunmap()` 函数，将 `(pte = walk(pagetable, a, 0)) == 0` 的情况由引发 panic 改为 continue 跳过。

```

# kernel/vm.c
void
uvmunmap(pagetable_t pagetable, uint64 va, uint64 size, int
do_free) //第二次修改后的 uvmunmap()
{
    uint64 a;
    pte_t *pte;

    va=PGROUNDDOWN(va);
    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + size*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0)
            //panic("uvmunmap: walk"); //L2 或 L1 中的 PTE 未分配, 致使 L0 页
            //目录就不存在, 即虚拟地址对应的 PTE 未分配
            continue;
        if((*pte & PTE_V) == 0) {
            continue;
            //panic("uvmunmap: not mapped");
        }
        if(PTE_FLAGS(*pte) == PTE_V)
            panic("uvmunmap: not a leaf");
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}

```

3. 而对于 `uvmcopy()` 的处理和 `uvmunmap()` 是一致的, 只需要将 PTE 无效和不存在两种情况由引发 `panic` 改为 `continue` 跳过即可。

```

int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    char *mem;
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0) //PTE 不存在
            continue;
        //panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0) //PTE 无效时，不产生 panic，而是跳过
            continue;
        //panic("uvmcopy: page not present");
        .....
    }
    .....
}

```

4. 在页面故障的中断处理过程中，若虚拟地址超过 $p \rightarrow sz$ （当前进程数据段末尾地址）或低于用户栈，则杀死进程。

```

# kernel/trap.c
void
usertrap(void) { //第二次修改后的 usertrap 函数
    .....
    else if (r_scause() == 13 || r_scause() == 15) {
        char *pa;
        uint64 va = r_stval();
        if(va >= p->sz){ // va 超过 p->sz
            p->killed = 1; //杀死进程
            goto end; //增加的 end flag 即 exit(-1)
        }
        if(va < PGROUNDUP(p->trapframe->sp)) { // va 低于用户栈
            p->killed = 1; //杀死进程
            goto end;
        }
        if ((pa = kalloc()) == 0) { //物理页分配失败
            printf("usertrap(): kalloc() failed\n");
            p->killed = 1;
            goto end;
        }
        memset(pa, 0, PGSIZE);
        if (mappages(p->pagetable, PGROUNDDOWN(va), PGSIZE, (uint64)
pa, PTE_W | PTE_R | PTE_U) != 0) {
            kfree(pa);
            printf("usertrap(): mappages() failed\n");
            p->killed = 1;
            goto end;
        }
    } else if((which_dev = devintr()) != 0){ }
    else {
        printf("usertrap(): unexpected scause %p pid=%d\n",
r_scause(), p->pid);
        printf("                sepc=%p stval=%p\n", r_sepc(),
r_stval());
        p->killed = 1;
    }
end:    // end flag
    if(p->killed)
        exit(-1);
    .....
}

```

5. 在 `walkaddr()` 函数的原本逻辑中, PTE 无效、PTE 不存在以及 PTE_U 标志位为 0 都会返回 0 表示失败。但是在实现了懒分配的情况下, PTE 无效和不存在是可以被允许的, 因此要对这两种情况进行处理。但并不是两种情况的所有情形都被允许, 由于懒分配是针对用户堆空间的, 因此还需要判断虚拟地址 `va` 是否在用户堆空间的范围内。

若满足范围要求, 则考虑如何分配内存。与前面不同的是, `read` 和 `write` 为系统调用, 执行到此处时已经在内核模式, 因此这里不会引发 `page fault`。这里和 `usertrap()` 中的处理类似, 根据虚拟地址分配对应的物理页。

```

# kernel/vm.c
uint64
walkaddr(pagetable_t pagetable, uint64 va) { // 修改后的 walkaddr 函数
    pte_t *pte;
    uint64 pa;
    struct proc *p = myproc(); // 获取当前进程
    if(va >= MAXVA) // 检查虚拟地址是否超过最大值
        return 0;
    pte = walk(pagetable, va, 0); // 获取虚拟地址 va 对应的页表项

    // 懒惰分配允许 PTE 无效和不存在
    if(pte == 0 || (*pte & PTE_V) == 0) {
        if(va >= PGROUNDUP(p->trapframe->sp) && va < p->sz){ // 如果虚拟地址位于用户堆栈中
            char *pa;
            // 分配一页物理内存
            if ((pa = kalloc()) == 0) {
                return 0; // 分配失败
            }
            memset(pa, 0, PGSIZE); // 将新分配的物理内存清零
            // 将虚拟地址 va 映射到分配的物理内存 pa
            if (mappages(p->pagetable, PGROUNDDOWN(va), PGSIZE,
                (uint64) pa, PTE_W | PTE_R | PTE_U) != 0) {
                kfree(pa); // 映射失败，释放分配的内存
                return 0;
            }
        } else {
            return 0; // 虚拟地址不在用户堆栈范围内
        }
    }
    if((*pte & PTE_U) == 0) // 检查页表项是否允许用户态访问
        return 0;
    pa = PTE2PA(*pte); // 获取页表项对应的物理地址
    return pa;
}

```